RESOLUÇÃO DE PROBLEMAS POR ME DE BUSCA

parte 1

Prof. Tacla UTFPR/Curitiba

BUSCA

Quais processos as pessoas usam quando solucionam problemas?

Pergunta inicial feita por Newell & Simon em suas pesquisas sobre estratégias de solução de problemas em experimentos de ciências Cognitivas com humanos – **Human Problem Solving** (HPS)

Hipótese de N&S sobre o que chamaram de busca heurística:

As pessoas podem solucionar problemas não-vistos anteriormente porque podem escolher entre ações alternativas, antecipar seus efeitos (resultados), avaliar seus resultados, voltar atrás e mudar a abordagem quando a avaliação é desfavorável.

BUSCA

Newell e Simon formalizaram sua teoria de HPS...

A busca 'heurística' ocorre mentalmente no espaço do problema que é um conjunto de possíveis soluções que o solucionador pode considerar.

A especificação do problema tem os seguintes componentes:

- representação mental do problema
- o objetivo a ser alcançado
- conjunto de ações que o solucionador utiliza para solucioná-lo

Uma estratégia de busca por solução é uma coleção de *heurísticas:*

- *podem ser preferências (condição-ação: em tal situação uso tal ação)*
- *podem ser uma* ***avaliação de estado*** *(continuo nesta direção ou não?)*

Uma solução é obtida pela aplicação de uma estratégia de busca no espaço do problema. A solução é um caminho – sequência de ações – do estado inicial até o estado objetivo.

Tópicos

1. Agentes para resolução de problemas
2. **Formulação** de problemas
3. Exemplos de problemas
4. **Soluções** aos problemas por busca
5. **Implementação** e **Avaliação**

*Capítulo 3 (Russel & Norvig)

Resolução de problemas por meio de buscas

AGENTES PARA RESOLUÇÃO DE PROBLEMAS

Exemplo: Romênia

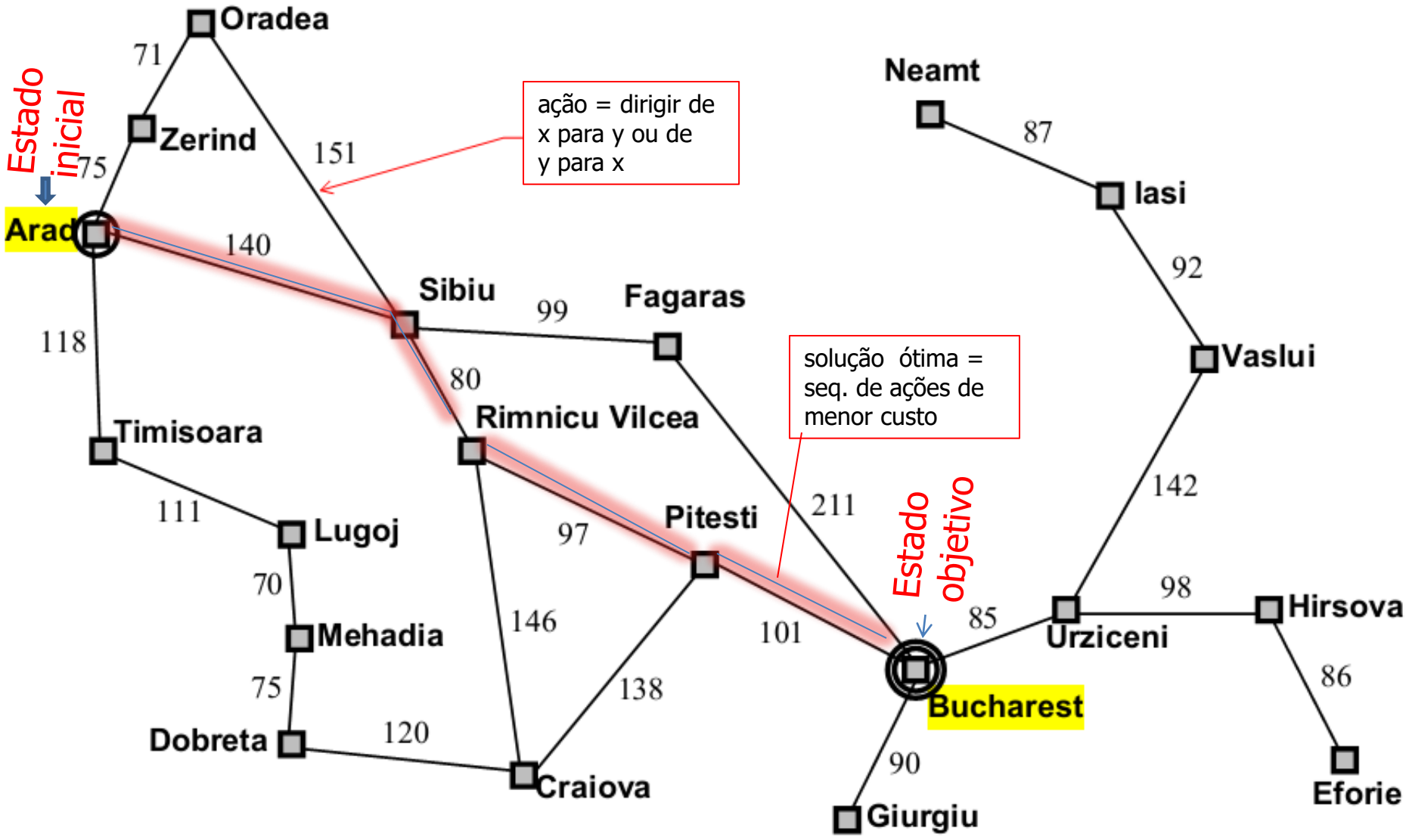
- Contexto:
  - De férias na Romênia, estamos em Arad
    - *(e queremos ver mais do país)*.
  - Temos voo marcado para amanhã, saindo de **Bucareste**.

- Formular objetivo:
  - estar em Bucareste

- Formular problema (definir abstrações relevantes)
  - **estados**: {Arad, Timisoara, Zerind,…} *// estar em Arad, …*
  - **ações**: Ir de Arad para Sibiu, Ir Sibiu para Fagaras, …

- Achar solução:
  - **sequência de ações**, e.g., ir(Sibiu), ir(Fagaras), ir(Bucareste)

Exemplo: Romênia



FORMULAÇÃO DO PROBLEMA E DA SOLUÇÃO

- **Estado inicial:** *Em(ARAD)*

- **Ações possíveis:** dado um estado *S*, quais ações são executáveis/aplicáveis em ***s***
  - ***ações(S): S → A***
  - **ações(ARAD)** = **{**IrPara(ZERIND), IrPara(TIMISOARA), …**}**

- **Modelo de transição ou Função sucessora**
  - **suc(s,a) : (s,a) → s'**
  - Transição de um estado **s** para um estado **s'** pela ação a
  - ex. `suc(ARAD, IrPara(ZERIND)) = ZERIND`

- **Espaço de estados do problema**
  - É dado pelo: **estado inicial + modelo de transição**
  - É o conjunto de todos os estados alcançáveis a partir do estado inicial através de qualquer sequência de ações
  - **GRAFO**

PROBLEMAS E SOLUÇÕES BEM FORMULADOS

- **Teste de objetivo**
  - Atingiu o objetivo?
  - Explícito*: Em(BUCHAREST)* ?
  - Implícito: XequeMate(x)

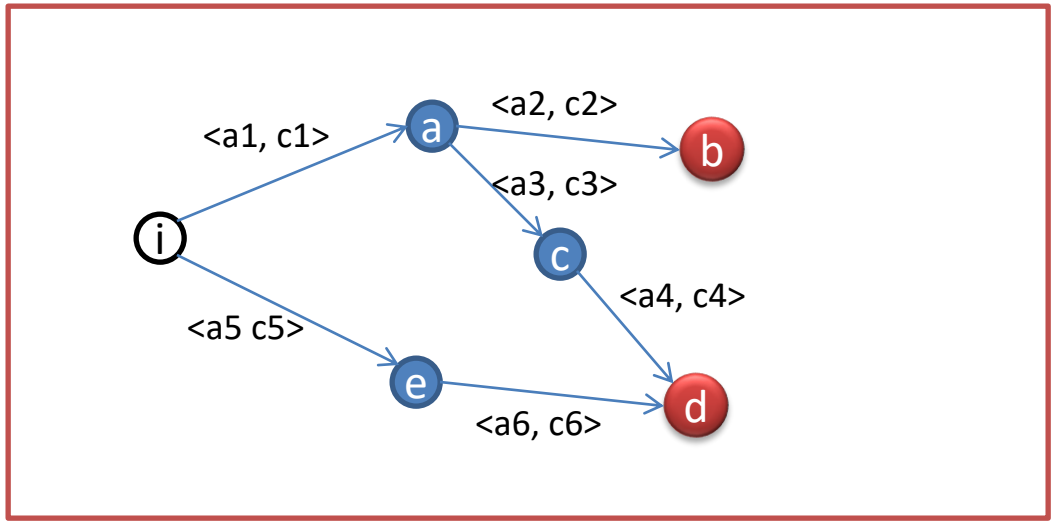
- **Custo do caminho (aditivo)**
  - ex. soma das distâncias, número de ações executadas, etc.
  - **c(s, a, s')** é o custo do passo/transição, supõe-se ≥ 0

- **Solução**
  - é a sequência de ações que levam do estado inicial a um estado objetivo
  - Ex. Ir(Sibiu) > ir(Fagaras) > ir(Bucharest)

- **Solução ótima**
  - é a solução de **menor custo (caminho)** entre todas as soluções.

ESPAÇO DE ESTADOS DO PROBLEMA

- Espaço de estados: comumente representado por um **grafo** no qual
  - um vértice representa um estado possível
  - um dos vértices é o estado inicial
  - um ou mais vértices representam estados-objetivos
  - arestas representam transições de estados resultantes de uma ação do agente
  - cada aresta tem um custo associado = custo da ação



b e d são estados objetivos
i é o estado inicial
<a, c> ação e custo

(não confundir com árvore de busca que é construída durante a execução do algoritmo de busca)

Agente baseado em objetivos

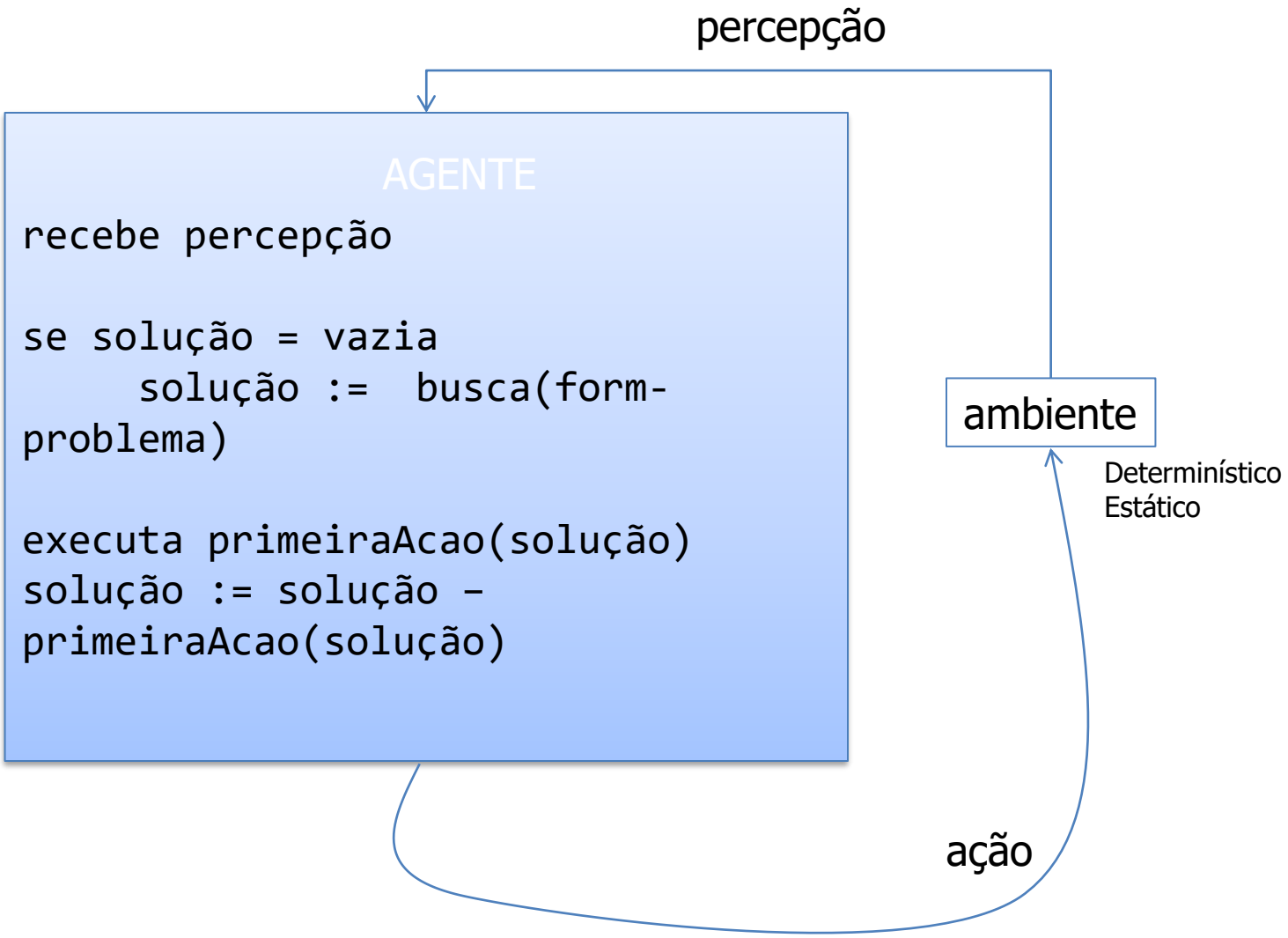
Vamos supor um agente em um ambiente estático e determinístico:

Como o ambiente é determinístico, quando o agente executa as ações não necessita tratar as percepções que dizem onde ele está a medida que ele se desloca ("tudo dá certo")

A **solução** é calculada no início da execução e é dada por uma sequência de ações.

A partir de uma percepção (ex. que diz onde o agente está) o agente consegue obter a solução (desde que já tenha um mapa do ambiente ou o ambiente seja completamente observável).

Agente baseado em objetivos



Agentes de Resolução de Problemas (*goal-based agents*)

função AG-SIMPLES-DE-RESOLUÇÃO-DE-PROBLEMAS(*percepção*)
retorna uma ação

variáveis estáticas: *seq*, uma seqüência de ações, inicialmente vazia
estado, alguma descrição do estado atual do mundo
objetivo, um objetivo, inicialmente nulo
problema, uma formulação de problema (dada)

estado := ATUALIZAR-ESTADO(*estado*, *percepção*)
se *seq* está vazia **então faça**
objetivo := FORMULAR-OBJETIVO(*estado*)
problema := FORMULAR-PROBLEMA(*estado*, *objetivo*)
seq := BUSCA(*problema*)
se *seq* = falha **então retornar** ação nula
ação := PRIMEIRO(*seq*)
seq := RESTO(*seq*)
retornar *ação*

Sistema de malha-aberta: o agente ignora as percepções que chegam do ambiente

***entradas**: *percepção*, uma percepção

Resolução de problemas por meio de buscas

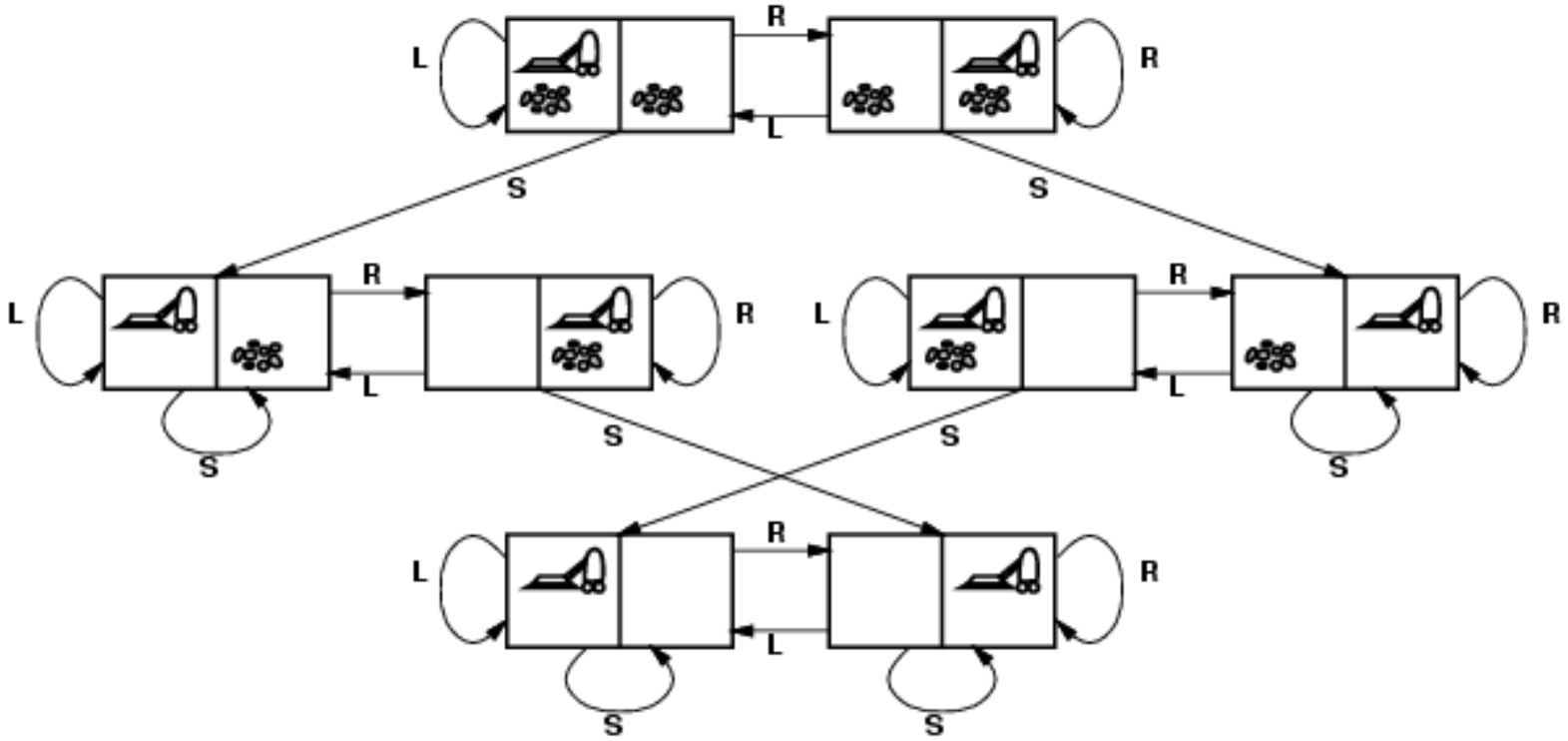
EXEMPLOS DE PROBLEMAS

Formulação de problemas

Nos slides que seguem, aborda-se a questão de como formular um problema.

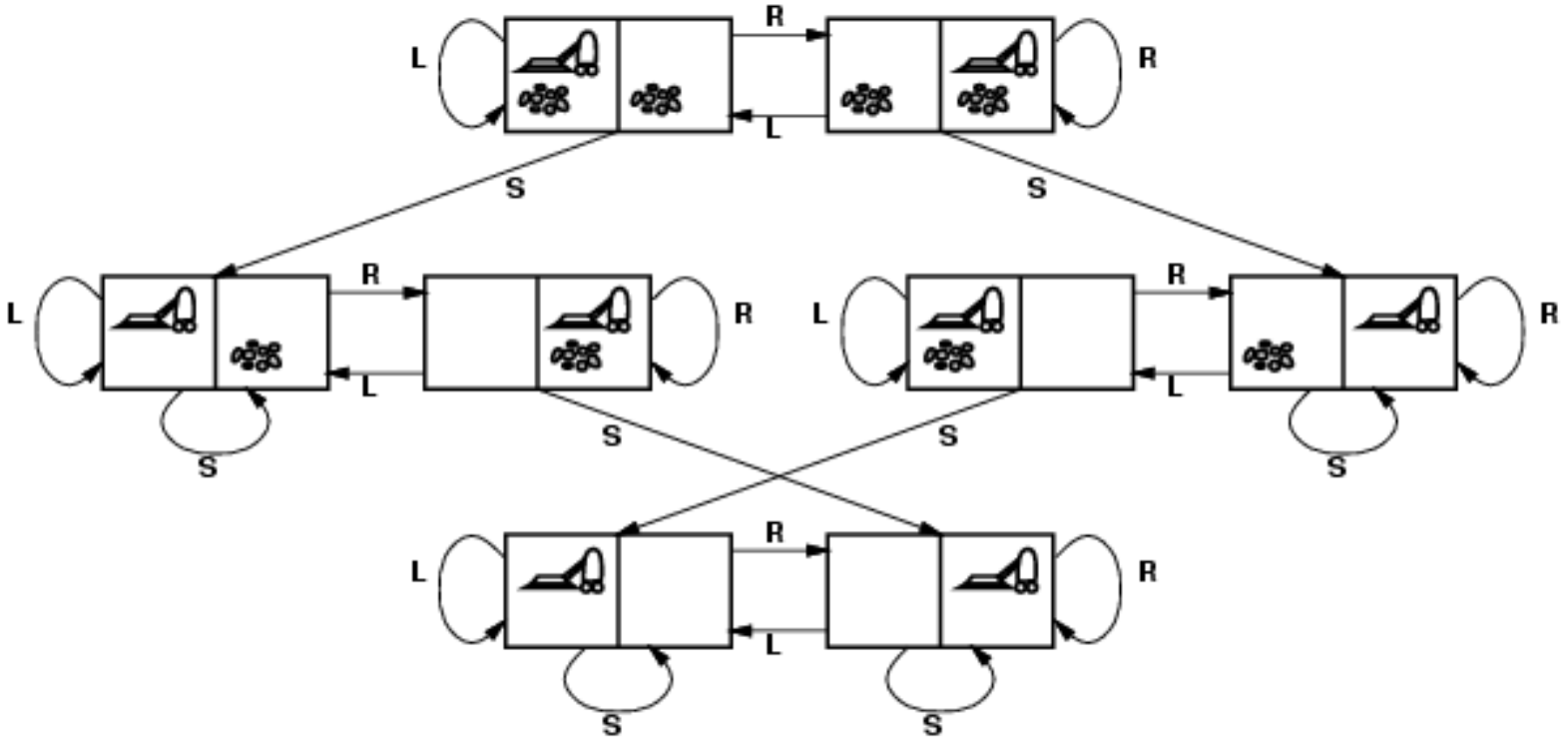
A escolha de uma boa abstração facilita a resolução do problema.

Grafo (espaço) de estados para o mundo do aspirador



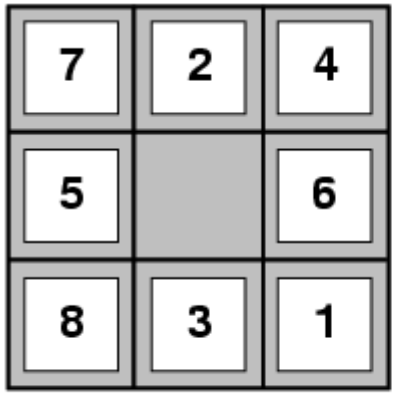
- estados?
- ações?
- estado inicial?
- testes objetivos?
- custo do caminho?

Grafo (espaço) de estados para o mundo do aspirador

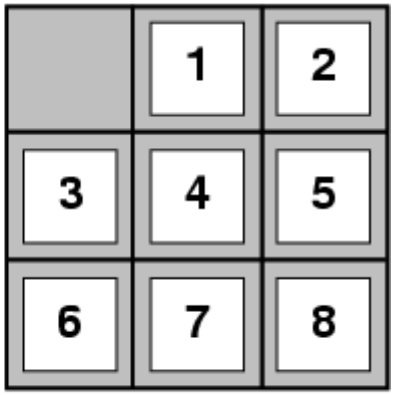


- estados? São 2 posições, cada posição com sujeira ou não: 2 x 2^2= 8
- ações? Left, right, suck
- estado inicial? Qualquer um dos 8 possíveis
- teste objetivo? Posição A e B limpas
- custo do caminho? 1 por ação executada

Exemplo: quebra cabeças de 8 peças



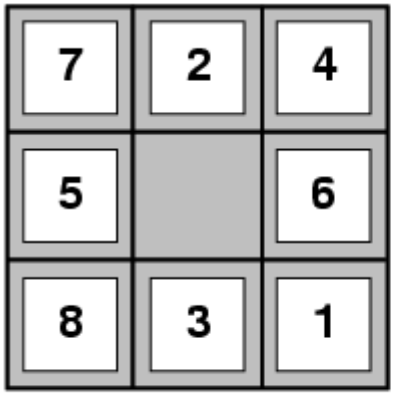
Estado Inicial



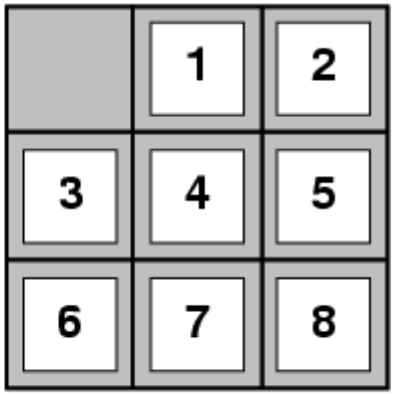
Estado Objetivo

- estados?
- ações?
- testes objetivos?
- custo do caminho?

Exemplo: quebra cabeças de 8 peças



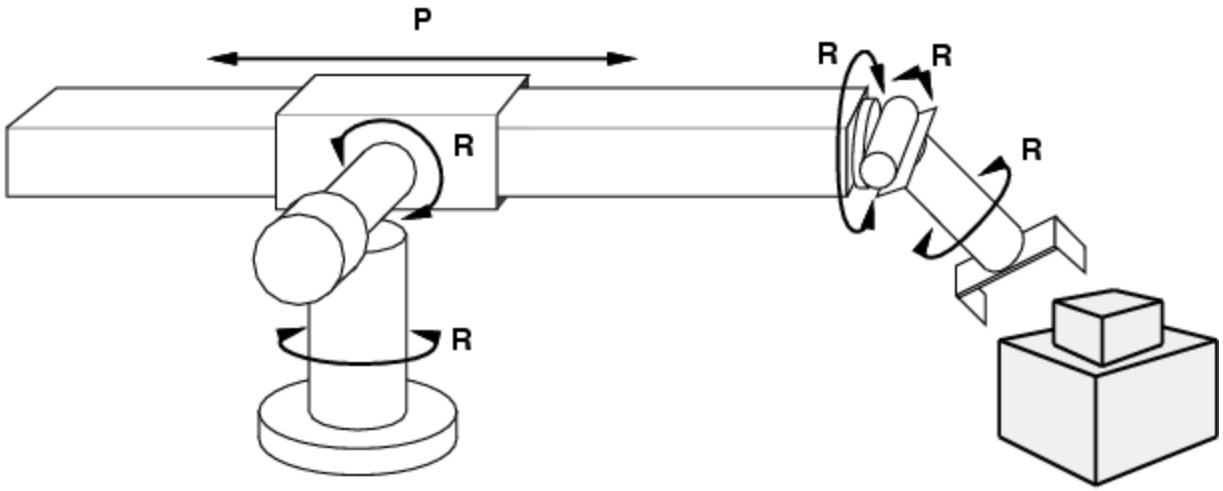
Estado Inicial



Estado Objetivo

- estados? Localização das peças
- ações? Mover o vazio para esq., dir., acima, abaixo
- Teste objetivo? Ver figura
- custo do caminho? 1 por ação

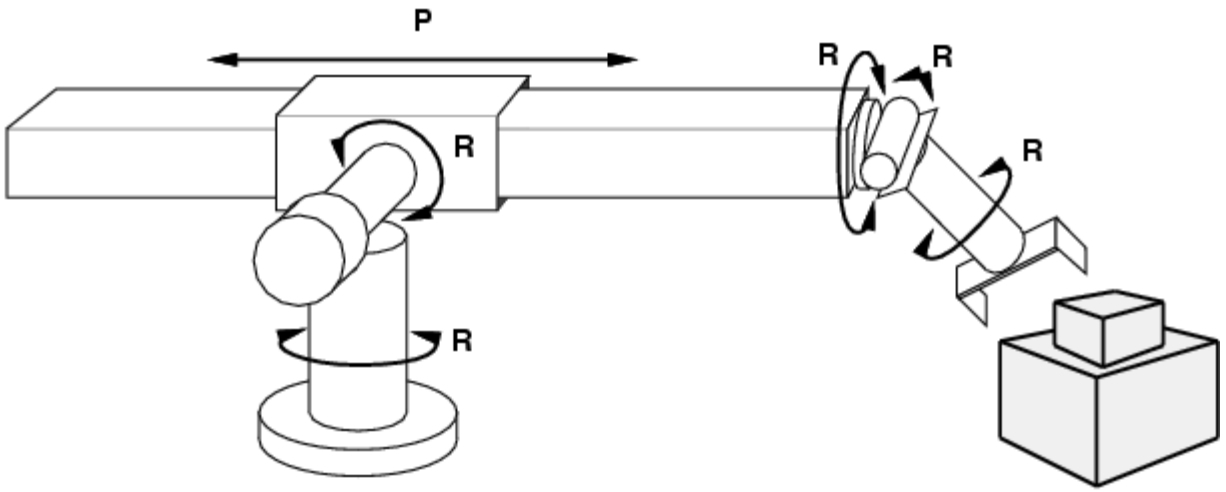
Exemplo: montagem robôtica



- Estados?

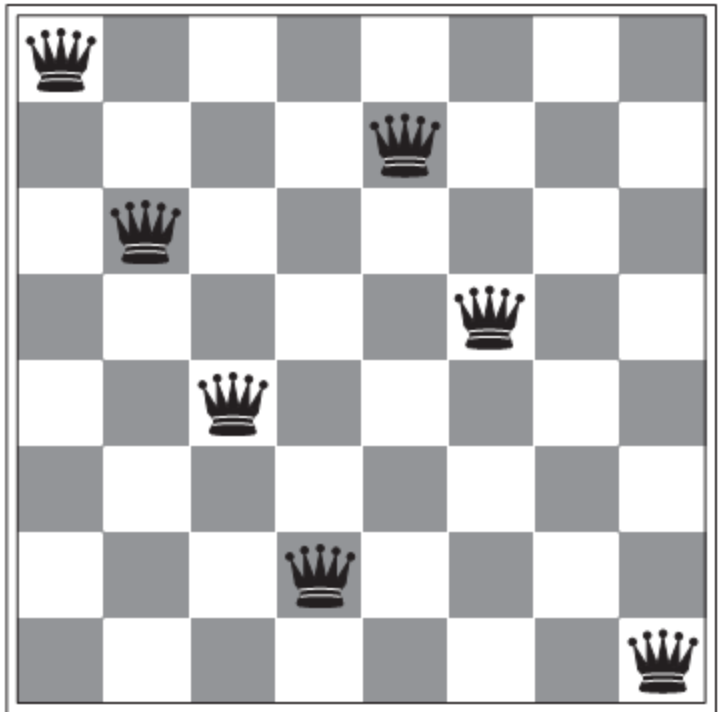
- Estado inicial?
- Ações?
- Teste objetivo?
- Custo do caminho?

Exemplo: montagem robôtica



- Estados? Coordenadas reais dos ângulos das juntas do robô e das peças do objeto a ser montado
- Estado inicial? Posição afastada do suporte
- Ações? Movimentações contínuas das juntas do robô
- Teste objetivo? Montagem completa da peça
- Custo do caminho? Tempo de execução

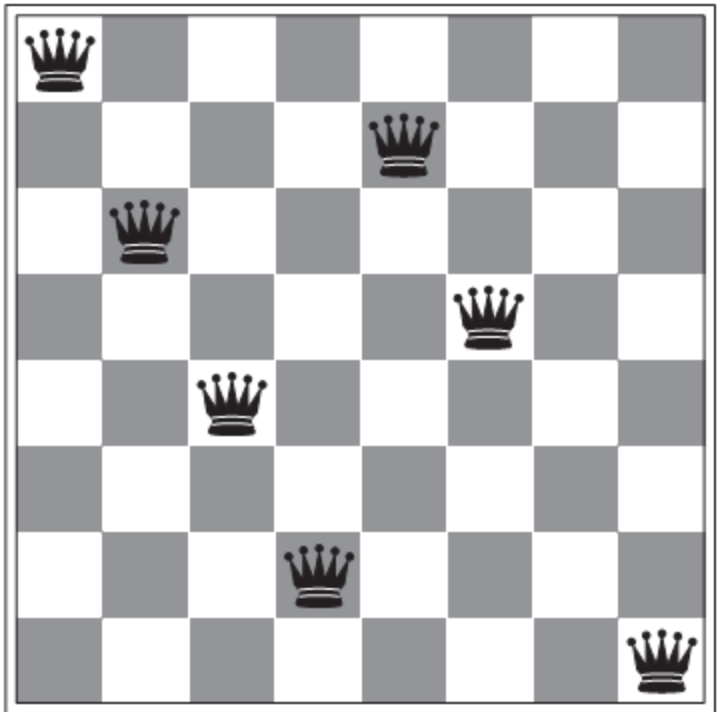
PROBLEMA DAS 8 RAINHAS



- estados?
- estado inicial?
- ações?

- testes objetivos?
- custo do caminho?

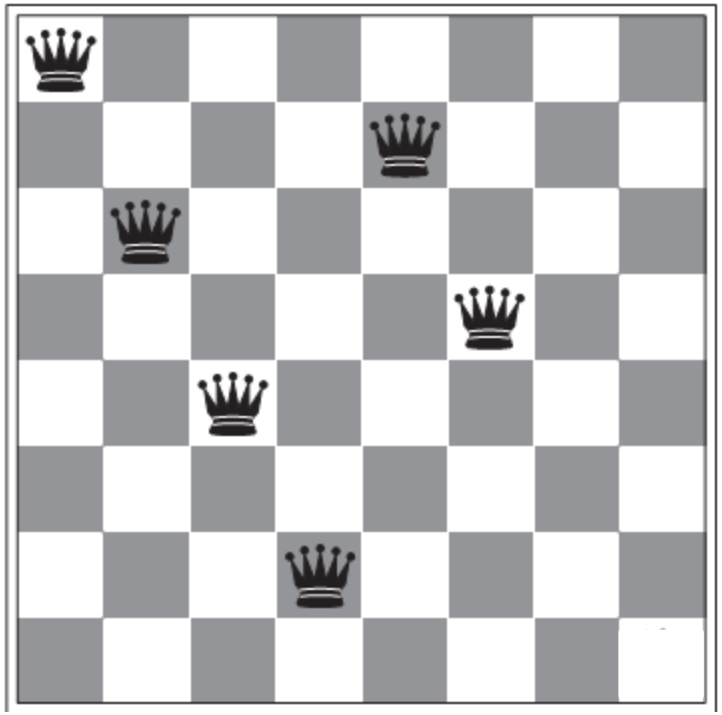
PROBLEMA DAS 8 RAINHAS: formulação 1



- estados? Qualquer arranjo contendo de 0 a 8 rainhas no tabuleiro
- estado inicial? Nenhuma rainha no tabuleiro
- ações? Adicionar uma rainha ao tabuleiro em qualquer posição vazia

- teste objetivo? 8 rainhas no tabuleiro e nenhuma atacada
- custo do caminho? Não interessa – interessa apenas o goal

PROBLEMA DAS 8 RAINHAS: formulação 2



estado possível:
6 rainhas nas colunas mais a esquerda sem nenhum ataque

- estados? Qualquer arranjo de *n* rainhas (0 <= n <= 8) , uma por coluna, nas *n* colunas mais a esquerda, com nenhuma rainha atacando outra
- estado inicial? Nenhuma rainha no tabuleiro
- ações? Adicionar uma rainha ao tabuleiro na n-ésima coluna mais a esquerda sem que nenhuma rainha ataque outra
- teste objetivo? 8 rainhas no tabuleiro e nenhuma atacada
- custo do caminho? Não interessa – interessa apenas o goal

PROBLEMAS DAS 8 RAINHAS

Este exemplo ilustra a importância de bem formular um problema:

Formulação 1a:

Colocar todas as rainhas simultaneamente no tabuleiro em qualquer posição.

Ação: trocar a posição de uma rainha por qualquer outra posição.

$$espaço\ de\ estados\colon\ C_8^{64} = \frac{64!}{8!\,56!} \approx 4.4\ x\ 10^9$$

Formulação 1b:

Colocar as 8 rainhas simultaneamente no tabuleiro sendo no máximo uma rainha por coluna.

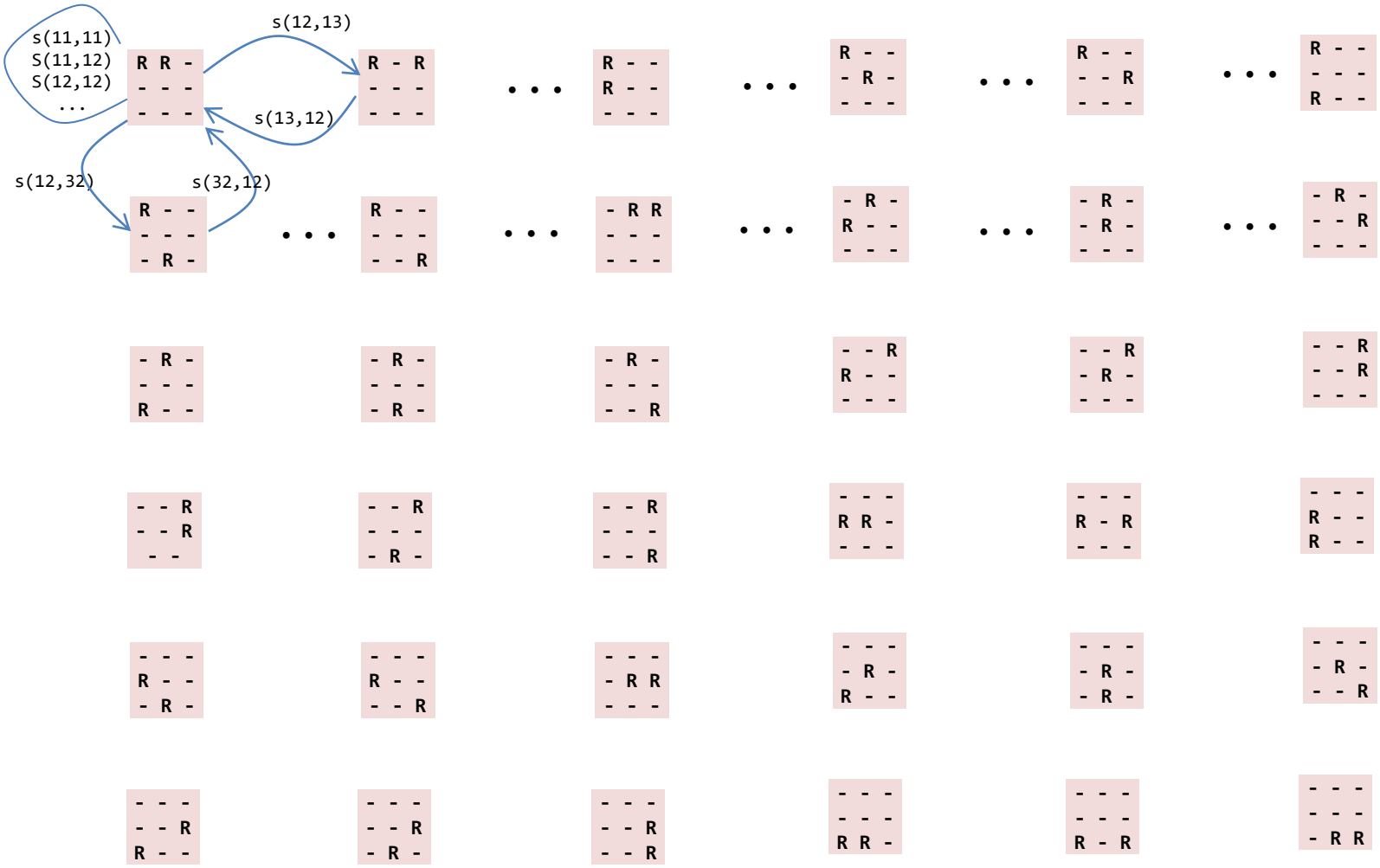
Ação: mover a posição de uma rainha para outra dentro da sua coluna

$$espaço\ de\ estados\ = 8^8$$

Formulação (1a): 3 x 3 com 2 rainhas

Colocar todas as rainhas – ação = swap
ação: swap(this, with) (índice: <lin><col>)

$$C_2^9 = \frac{9!}{2!\,7!} = 36$$



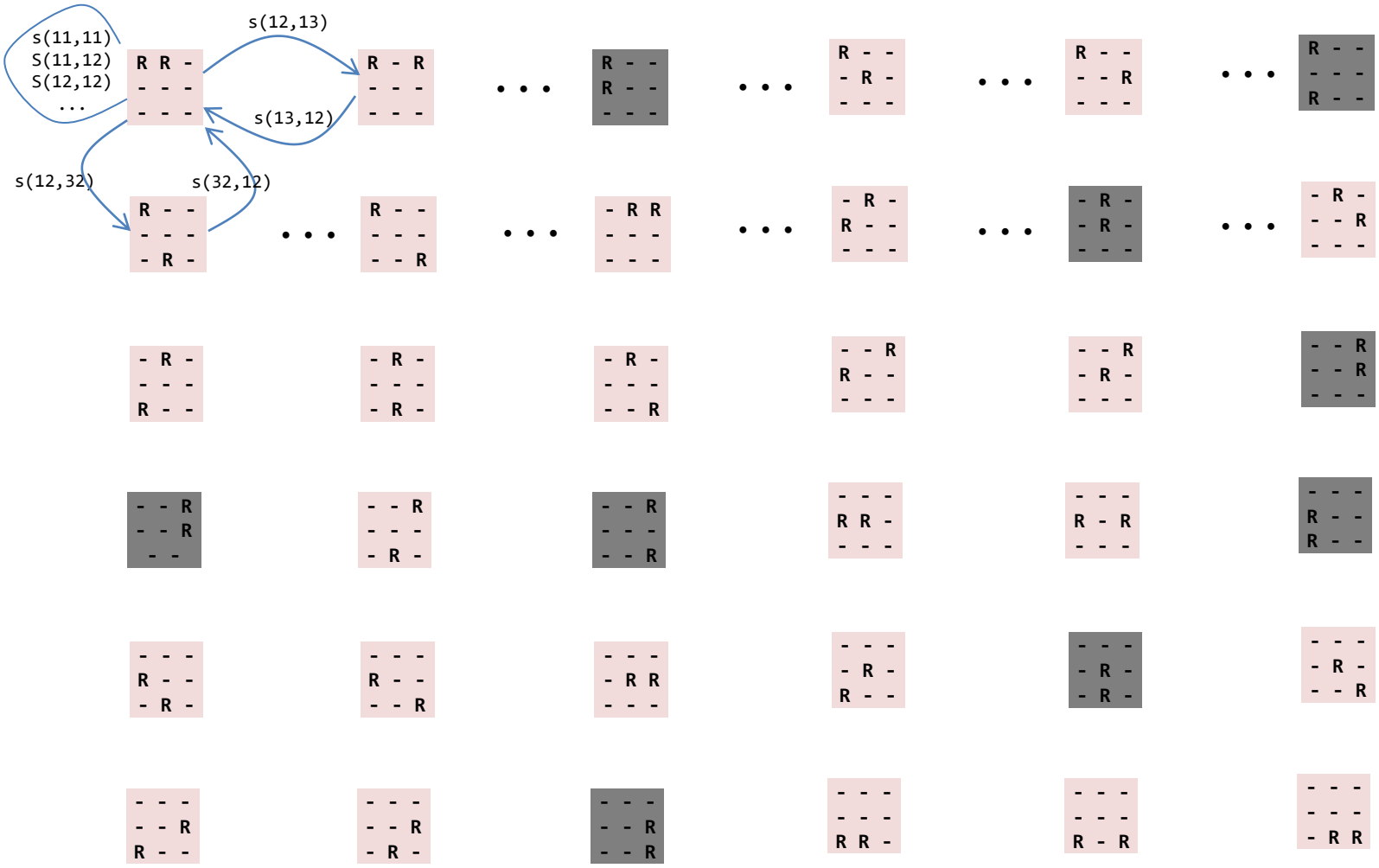
Formulação (1b): 3 x 3 com 2 rainhas

Colocar todas as rainhas – ação = swap

Restrição: 1 rainha por coluna

ação: swap(this, with) (índice: <lin><col>)

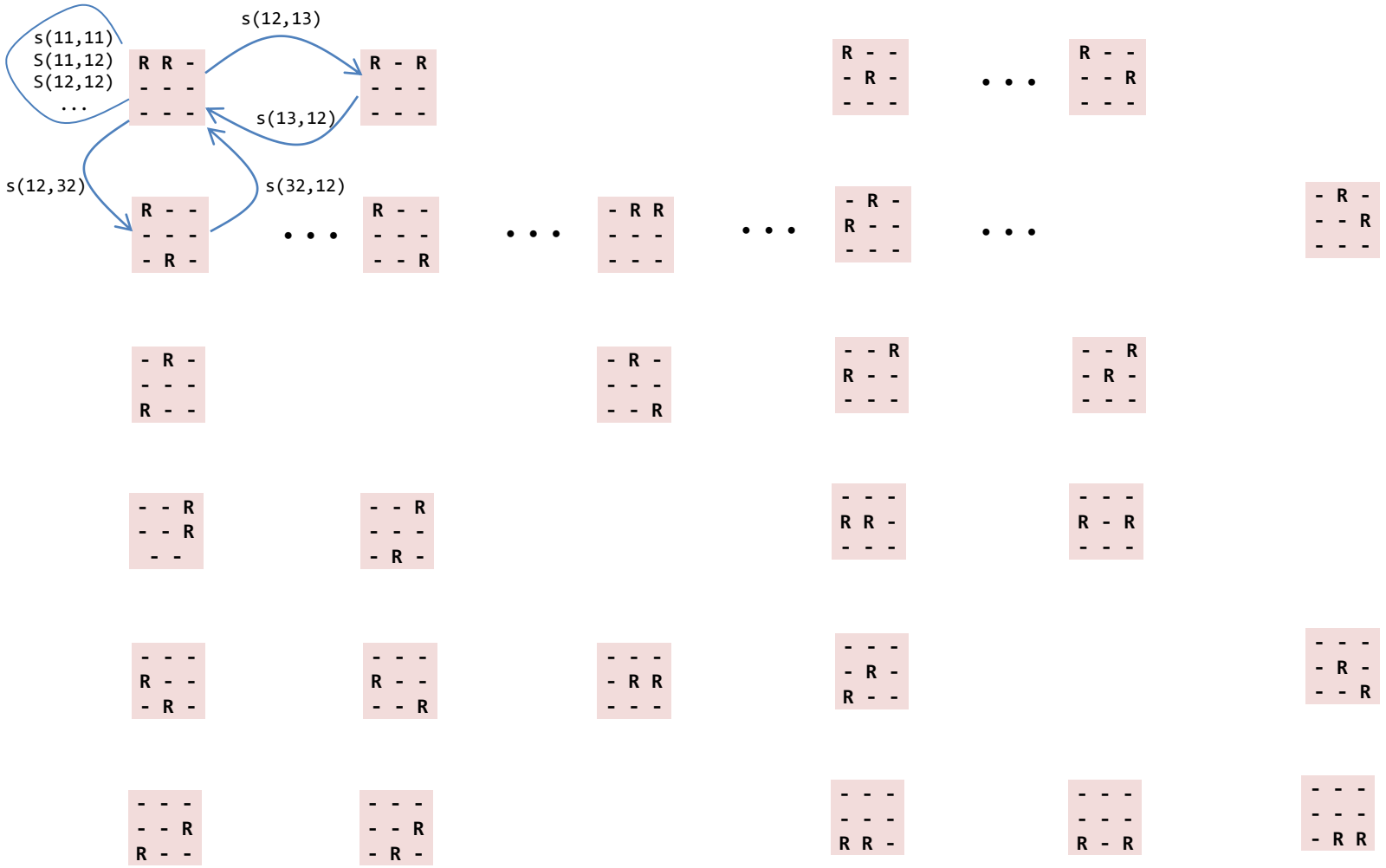
$36 - 9 = 27$



Formulação (1b): 3 x 3 com 2 rainhas

Colocar todas as rainhas – ação = swap
Restrição: 1 rainha por coluna
ação: swap(this, with) (índice: <lin><col>)

esp = 3 x 9 = 27



PROBLEMAS DAS 8 RAINHAS

Formulação 2a:

Colocar as n=8 rainhas <u>incrementalmente</u> em qualquer posição vazia.

$$estados = C_0^{64} + C_1^{64} + \ldots + C_n^{64} \approx 5.13\ x\ 10^9$$

vazio 1 rainha n rainhas

Nesta formulação um dos estados pode ser todas as rainhas na mesma coluna (o que claramente não é um estado objetivo).

Formulação 2b:

Colocar as rainhas <u>incrementalmente sendo uma rainha por coluna</u>

$$estados = 8^8 \approx 1.7\ x\ 10^7$$

princípio multiplicativo

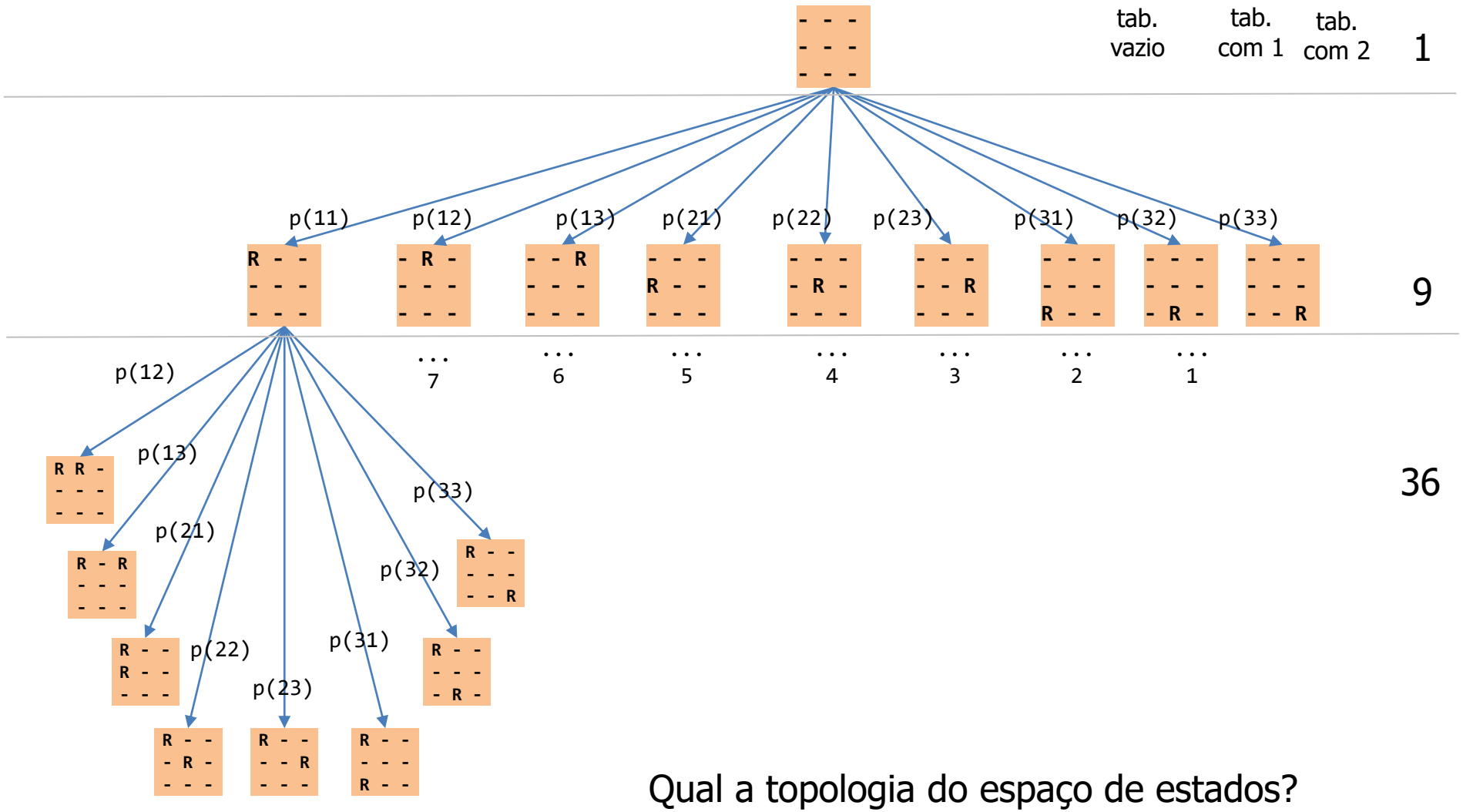
Nesta formulação um dos estados pode ser todas as rainhas na mesma linha (o que claramente não é um estado objetivo).

Formulação (2a): 3 x 3 com 2 rainhas

Colocar uma rainha INCR. EM QUALQUER POSIÇÃO VAZIA
ação: p(lc) por na <lin><col>

$$C_0^9 + C_1^9 + C_2^9 = 46$$

tab. vazio | tab. com 1 | tab. com 2



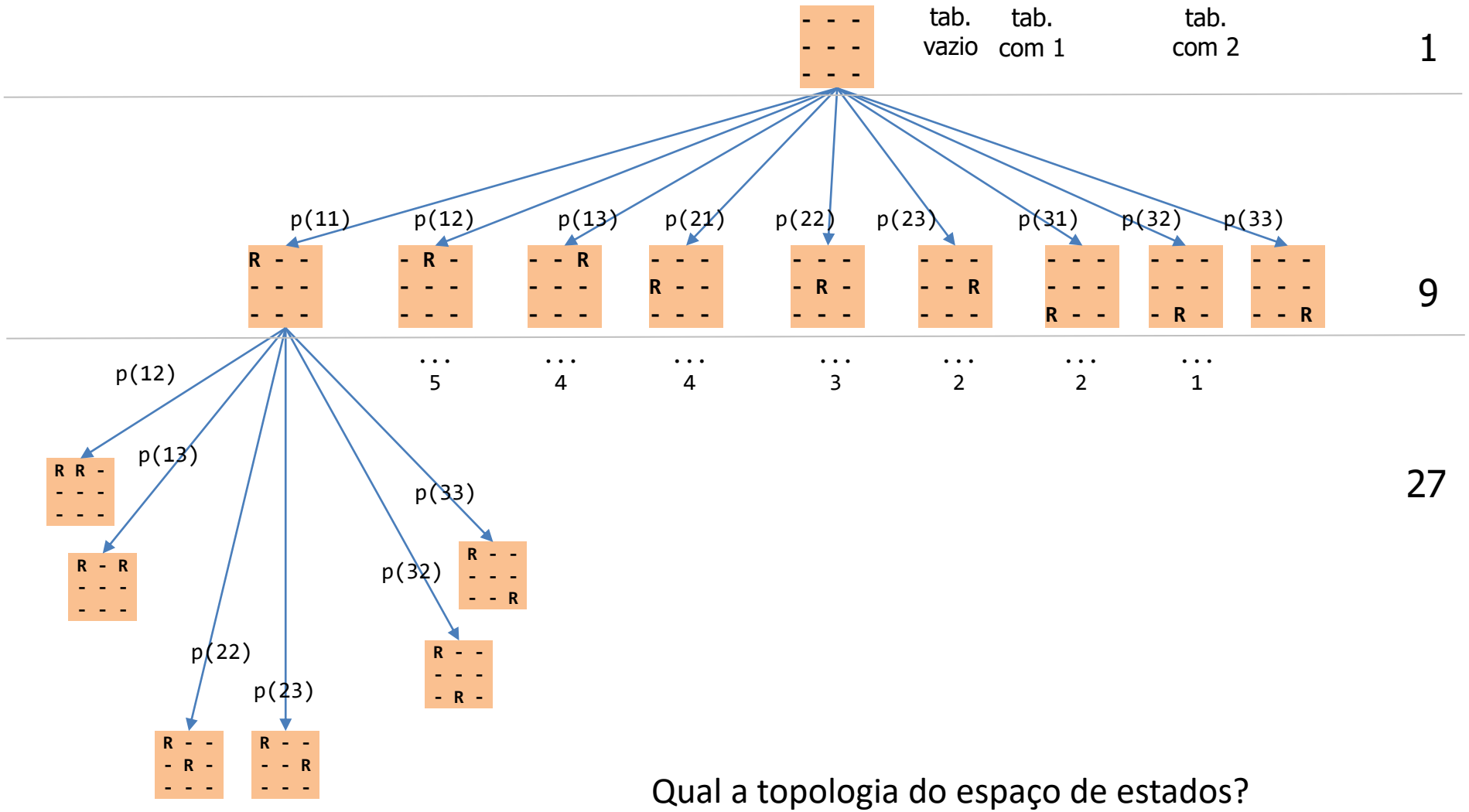
Qual a topologia do espaço de estados?
Quantos estados?

Formulação (2b): 3 x 3 com 2 rainhas

Colocar INCR. UMA RAINHA POR COLUNA
ação: p(lc) por na <lin><col>

$$C_0^9 + C_1^9 + (C_2^9 - restrições) = 37$$

tab. vazio | tab. com 1 | tab. com 2



Qual a topologia do espaço de estados?
Quantos estados?

PROBLEMAS DAS 8 RAINHAS

Formulação 3:

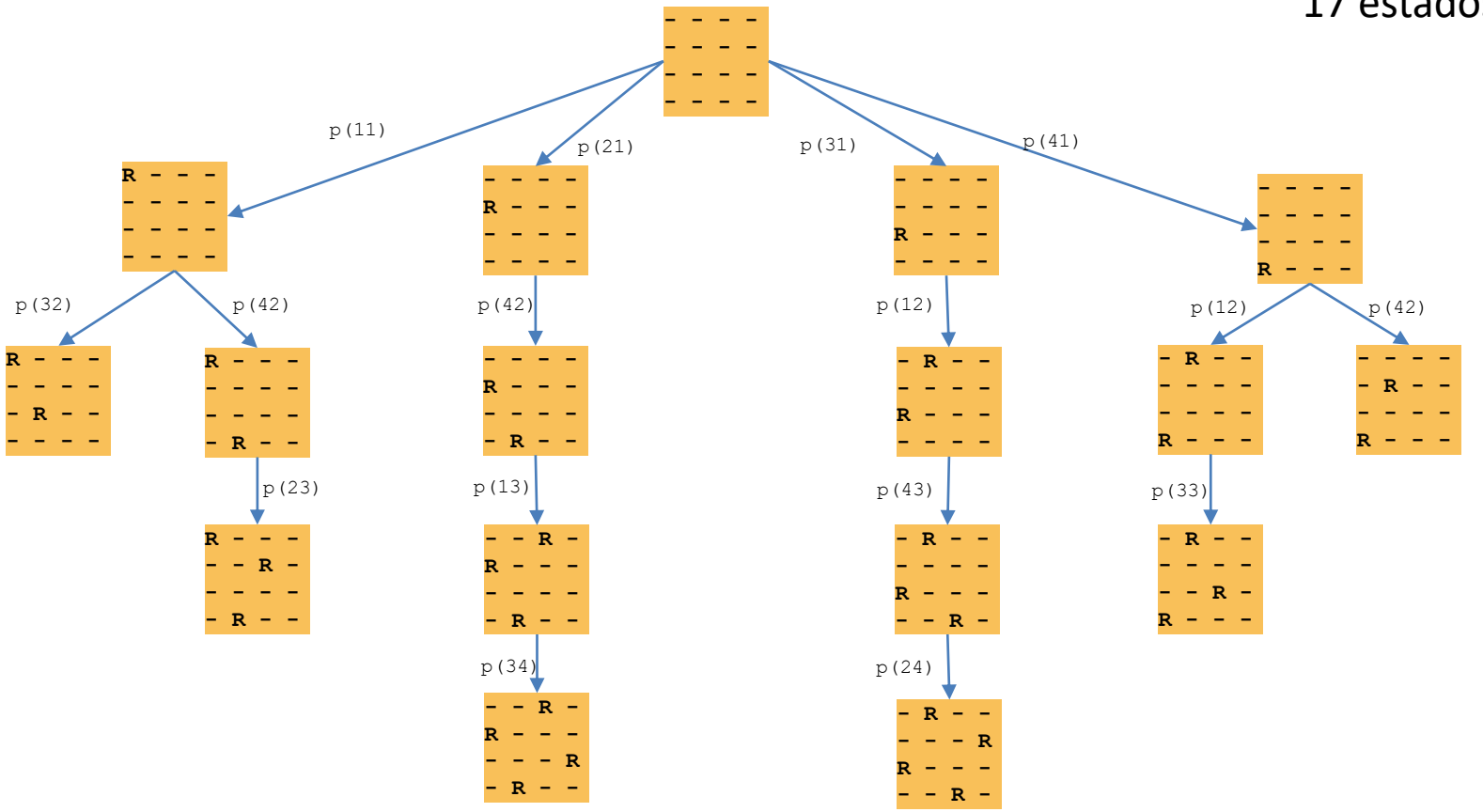
Colocar as rainhas incrementalmente sendo uma no máximo por coluna a partir da coluna mais a esquerda sem que ataque outra.

Para 8 rainhas: número de estados = 2.057

Formulação (3): 4 x 4 com 4 rainhas

Colocar uma rainha por coluna sem ataques
ação: p(lc)

17 estados



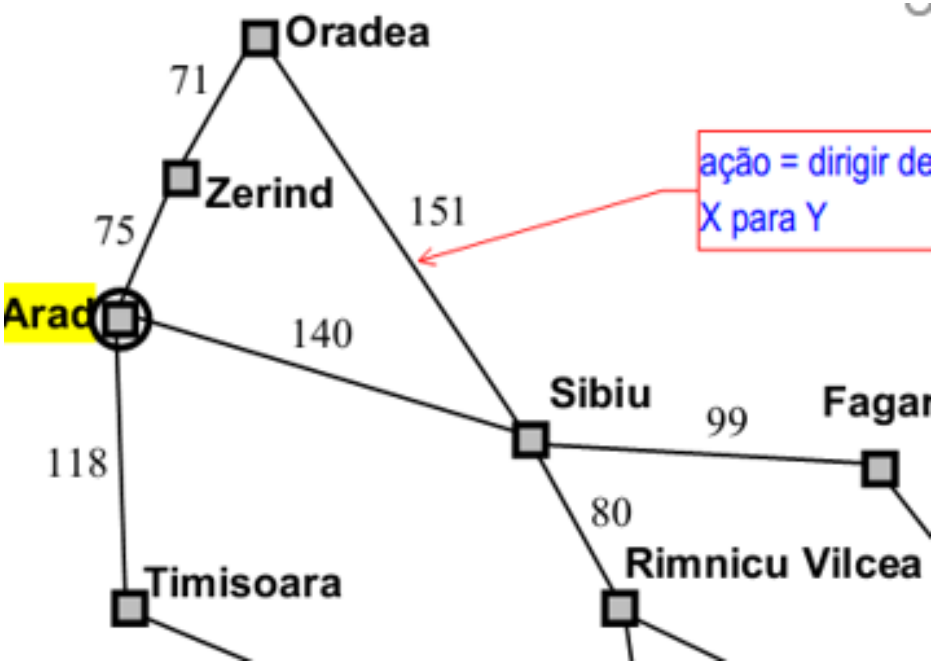
Qual a topologia do espaço de estados?
Quantos estados?

Resolução de problemas por meio de buscas

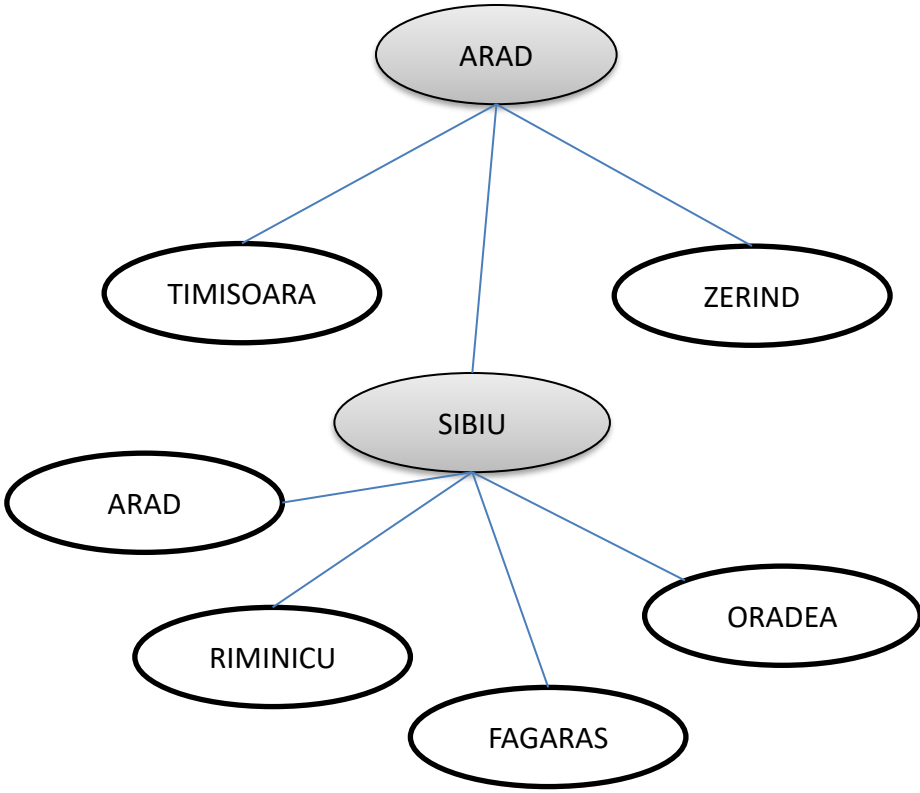
SOLUÇÕES AOS PROBLEMAS POR BUSCA

Espaço de estados x Espaço de Busca

Grafo: representa espaço de estados



Árvore: espaço de busca

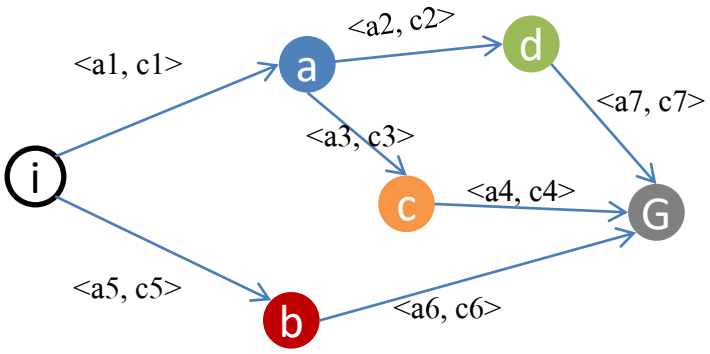


Grafo de estados x Árvore de busca (1)

O conjunto de todos os caminhos do nó inicial até os nós objetivos de um grafo de estados podem ser representados numa árvore (árvore é um grafo direcionado acíclico)

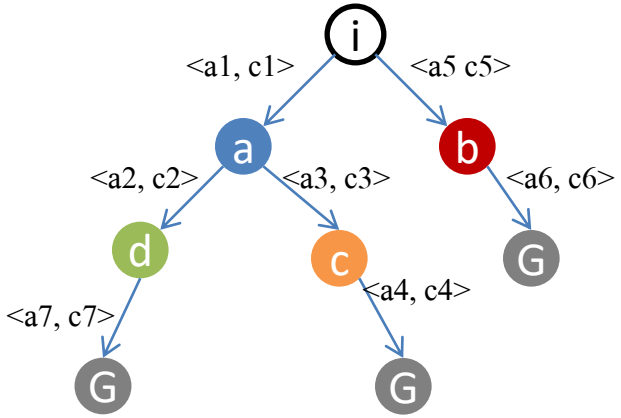
Grafos podem ser transformados em árvores por meio da **duplicação dos nós** e pela **eliminação de caminhos cíclicos** (se existirem e se forem tratados)

formulação: grafo



Espaço de estados do problema

busca de solução: árvore



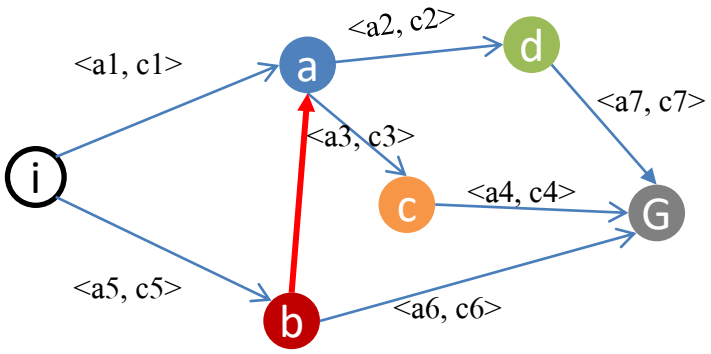
Árvore gerada por um algoritmo de busca exaustiva

Grafo de estados x Árvore de busca (2)

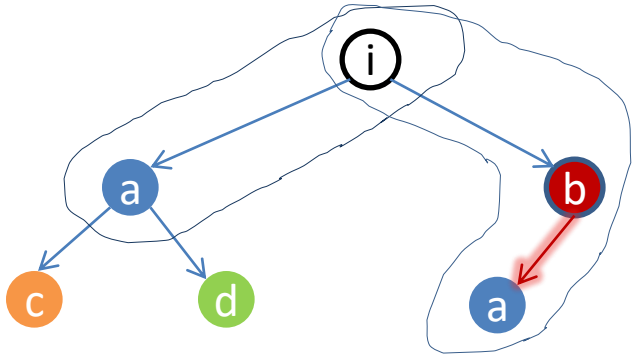
CAMINHOS REDUNDANTES NO ESPAÇO DE ESTADOS

Quando há mais de uma maneira de ir de um estado para outro. Caminhos redundantes no espaço de estados podem aumentar exponencialmente o **espaço de busca. As estratégias de busca tratam estas redundâncias de modos diferentes.**

formulação



busca de solução



Espaço de estados do problema (6 estados)

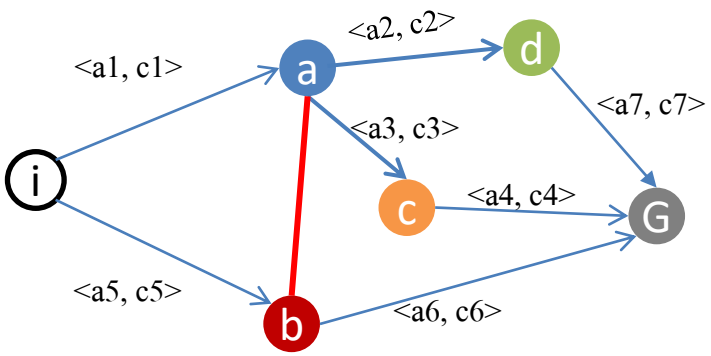
Árvore de busca em largura (parcial) (vale a pena colocar o estado 'a'?)

Ao incluir a aresta (b, a) geram-se caminhos redundantes entre *i* e *a*

Grafo de estados x Árvore de busca (3)

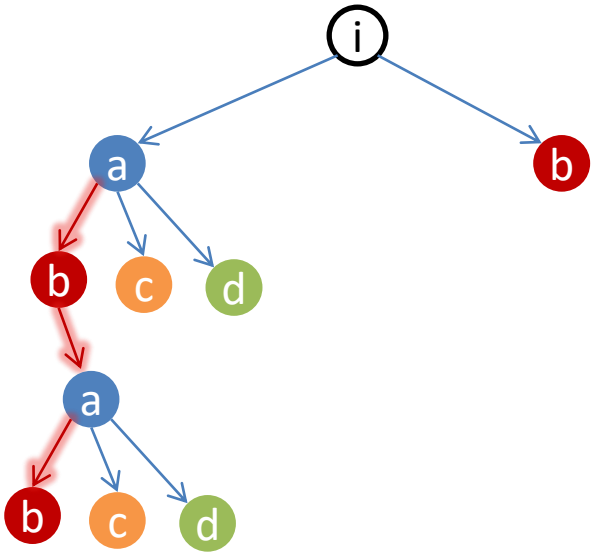
CAMINHOS REDUNDANTES originados de ciclos, laços e arestas não-direcionadas no espaço de estados. Se não controlados, podem levar à criação de um **espaço de busca infinito.**

formulação



Espaço de estados do problema
(6 estados)

busca de solução

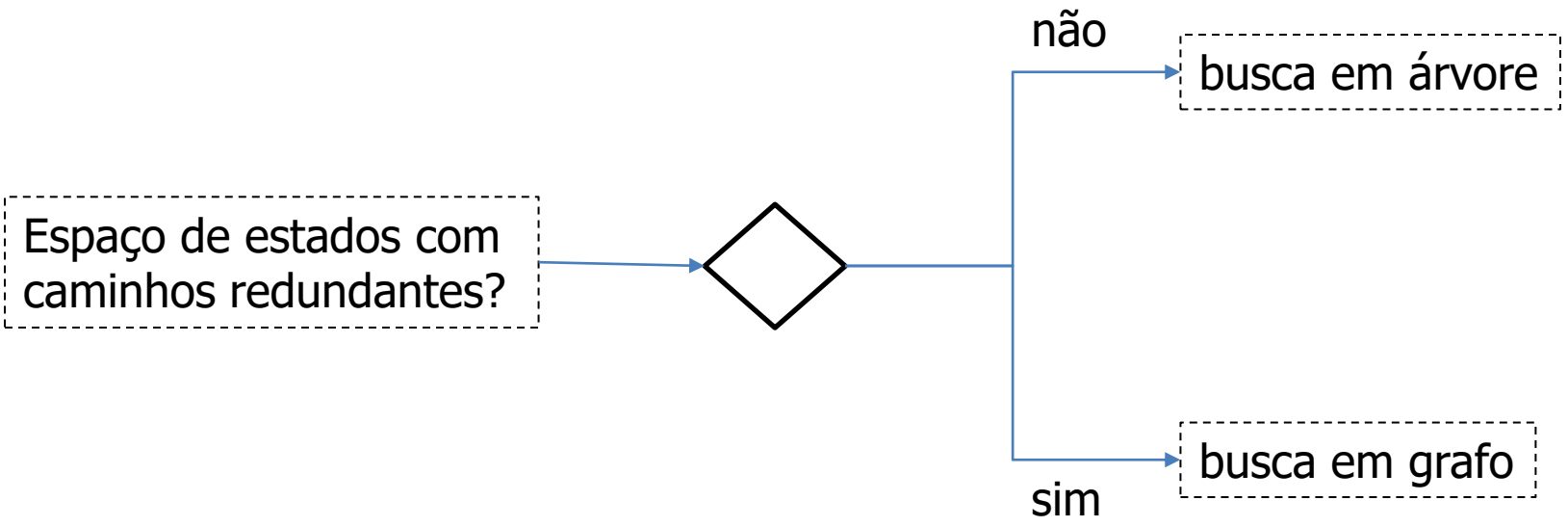


...

Árvore de busca estratégia profundidade
(infinitos nós)

Ao transformar a aresta (e, a) em não-direcionada, infinitos caminhos entre i e a podem ser gerados.

Algoritmos de busca



Algoritmos de busca em árvore

Idéia básica: explorar o espaço de estados de forma simulada (off-line), gerando-se estados sucessores (expandindo) até atingir o estado objetivo.

função BUSCA-EM-ÁRVORE(*problema*) **retorna** uma solução, ou falha

inicialize a *fronteira* da árvore usando o estado inicial do *problema*

repita

se a *fronteira* está vazia **então retornar** falha

escolha um nó folha para expansão e remova-o da *fronteira (estratégia)*

se o nó contém o estado objetivo **então retornar** a solução correspondente

expandir o nó e adicionar os nós resultantes à *fronteira* da árvore de busca

Algoritmos de busca em GRAFO

função BUSCA-EM-GRAFO(*problema*) **retorna** uma solução, ou falha

inicialize a *fronteira* da árvore usando o estado inicial do *problema*

inicialize *explorados* com vazio

repita

se a *fronteira* está vazia **então retornar** falha

escolha um nó folha para expansão e remova-o da *fronteira (estratégia)*

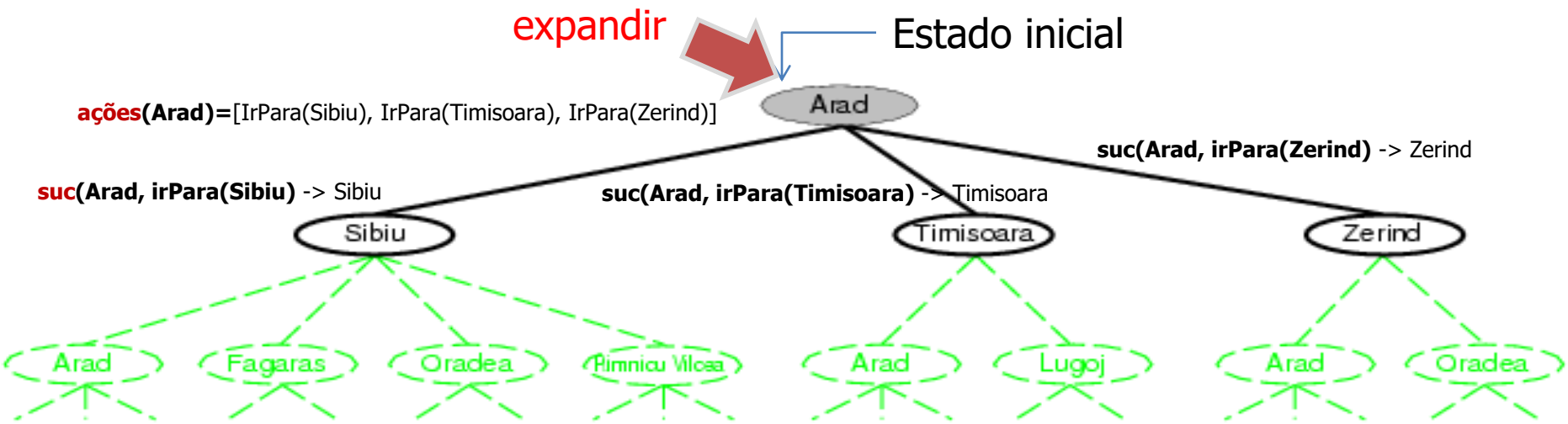
se o nó contém o estado objetivo **então retornar** a solução correspondente

adicione o estado aos *explorados*

adicionar cada um dos sucessores do nó à fronteira se somente se o estado correspondente não estiver em *explorados ou na fronteira*

Soluciona o problema de caminhos redundantes da busca em árvore por meio de uma lista de nodos já explorados (*também chamada de lista fechada)*

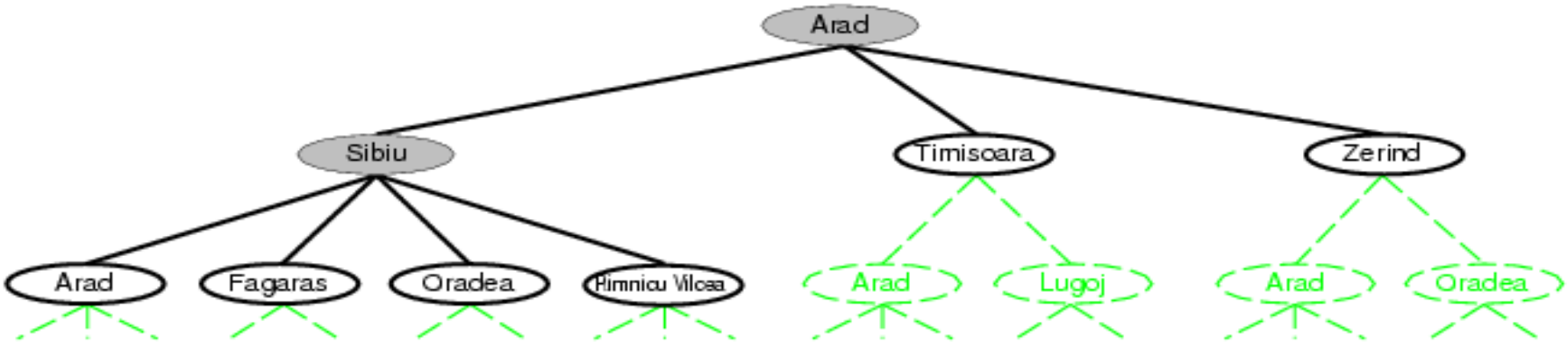
Exemplo de busca em grafo



Expandir o nó corrente resulta na árvore em negrito

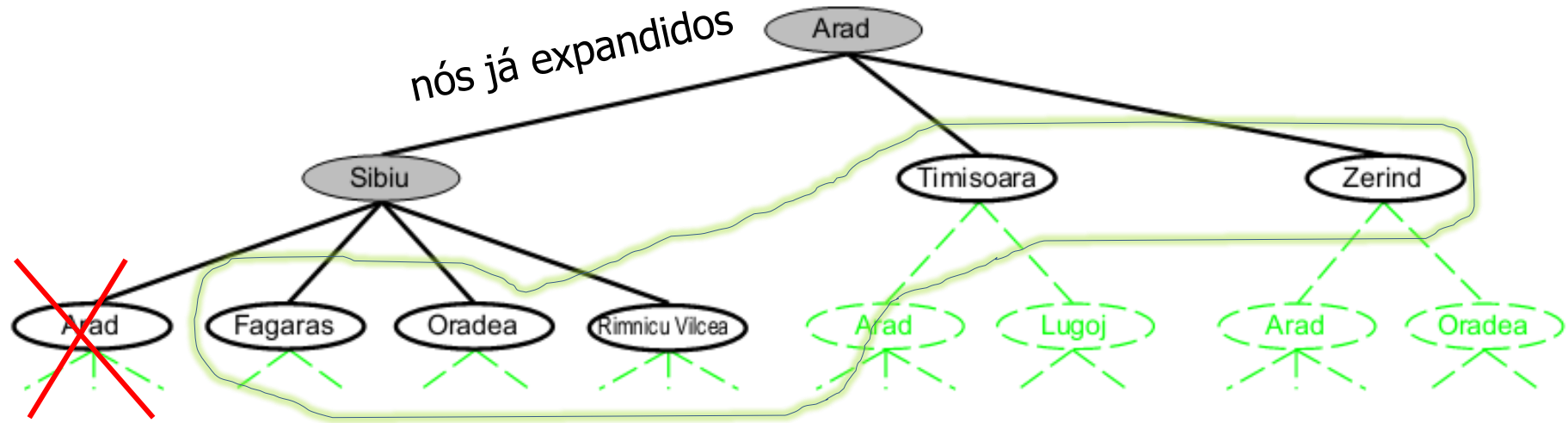
Árvore de busca parcial

Exemplo de busca em grafo



Árvore de busca parcial

Algoritmos de busca: nomenclatura



Controle de redundância

Fronteira da árvore de busca: são os nós folhas
num dado momento *(também chamada de borda ou lista aberta).*
Nós que representam estados já explorados podem ser eliminados da fronteira (ex. Arad)

Estratégia de busca: define qual nó da fronteira será o próximo a ser expandido

Implementação: estados vs. nós

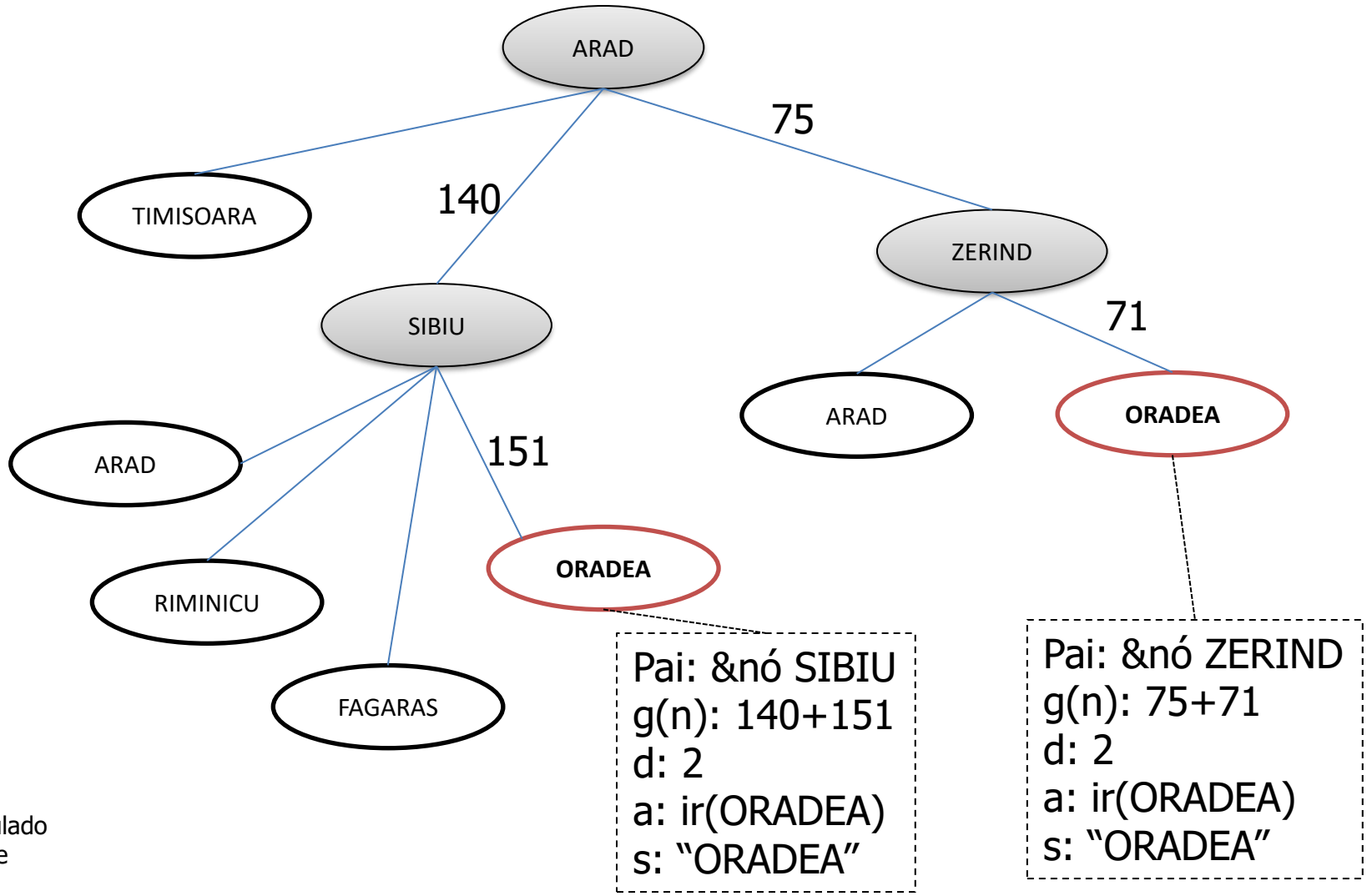
Estado: é a abstração de uma configuração do ambiente (ou mundo)

Nó: é uma estrutura de dados e **faz** parte de uma árvore de busca.

Logo, um estado pode ser representado por um ou mais nós em uma árvore de busca.

Implementação: estados vs. nós

Dois nós que apontam para o mesmo estado que foi alcançado por caminhos diferentes.



g = custo acumulado
d = profundidade
a = ação
s = label do estado

Estratégia de busca

- Uma **estratégia de busca** é definida pela escolha do próximo nó a ser expandido

- Logo, os algoritmos
  - busca em árvore
  - busca em grafo
  - representam uma família de algoritmos

AVALIAÇÃO DAS ESTRATÉGIAS DE BUSCA

Tipos de avaliação	Significado
Completa	garante encontrar **uma solução** se ela existe?
Ótima	garante encontrar a **solução de menor custo** (se existe solução)?
Complexidade de **tempo**	medido pelo número de nós gerados para executar a busca
Complexidade **espacial**	medido pelo **número máximo** de nós mantidos em memória em algum instante da execução da busca

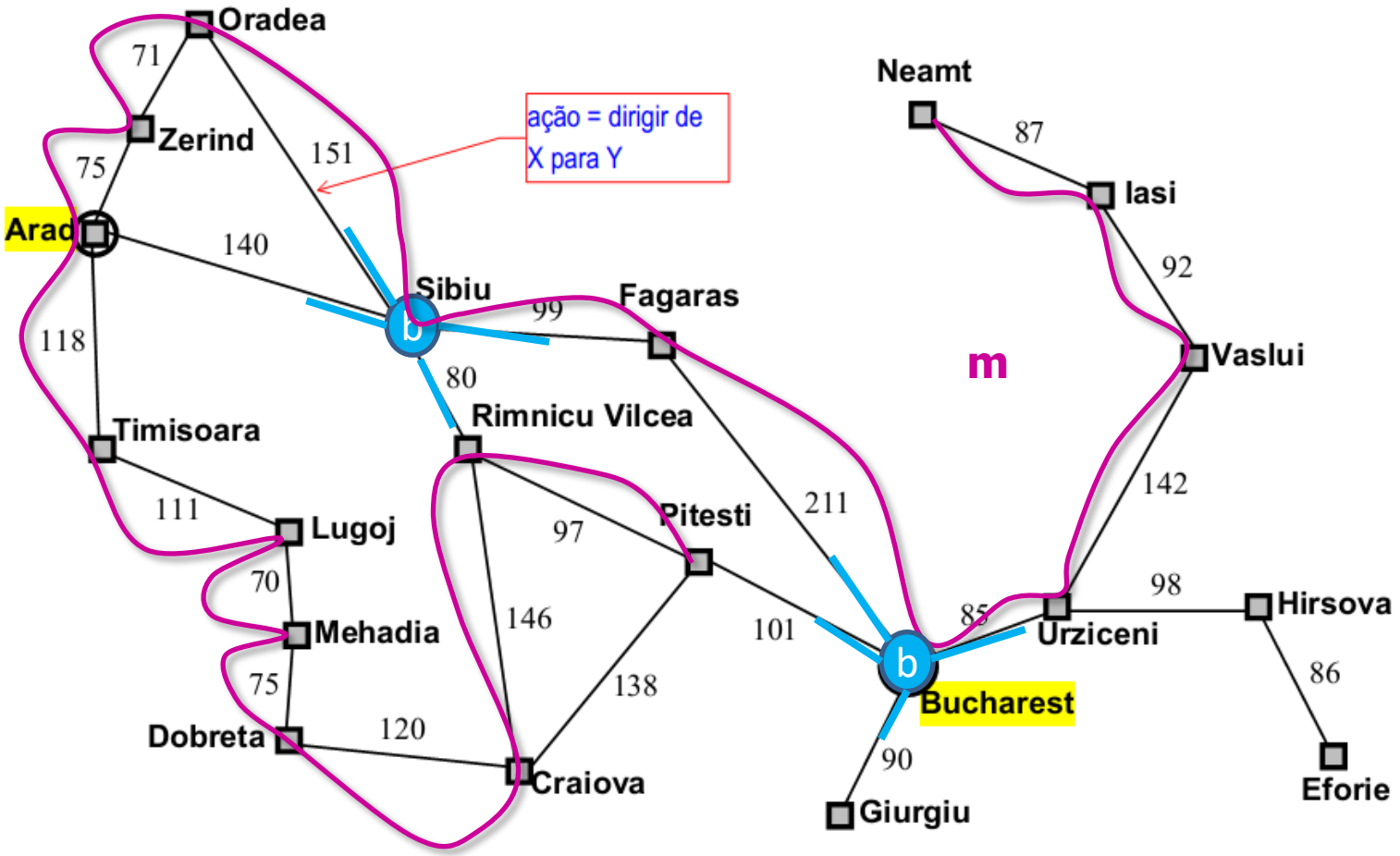
AVALIAÇÃO DAS ESTRATÉGIAS DE BUSCA

Variáveis que afetam as complexidades temporal e espacial

Variável	**Significado**
b = *branching* = ramificação	número máximo de sucessores (dentre todos os nós da árvore)
d = *depth* = profundidade	profundidade do nó objetivo mais raso (não é necessariamente o nó objetivo ótimo)
m (maximum len.)	tamanho máximo de caminho elementar entre todos os caminhos elementares do **espaço de estados** (pode ser ∞; não leva em conta o custo do caminho; corresponde à profundidade da árvore de busca*

caminho elementar: não passa duas vezes pelo mesmo vértice
*eliminando-se caminhos cíclicos

Exemplo das medidas



b = 4
d = 3
m = tamanho máximo de caminho =16

BUSCA

Simon and Newell (1971) stated the problem concisely:

"The initial question we asked in our research was:
'*What processes do people use to solve problems?'*

The answer we have proposed is:
'*They carry out selective search in a problem space that incorporates some of the structural information of the task environment.'*

Our answer now leads to the new question:

'How do people generate a problem space when confronted with a new task?' Thus, our research, like all scientific efforts, has answered some questions at the cost of generating some new ones." (Simon & Newell, 1971, p. 154)

Simon, H. A., & Newell, A. (1971). Human problem solving: The state of the theory in 1970. *American Psychologist*, *26*(2), 145–159. http://dx.doi.org/10.1037/h0030806